

INFO145 - Diseño y Análisis de Algoritmos

7 - Divide y Vencerás

Héctor Ferrada
académico

Instituto de Informática
Universidad Austral de Chile

Técnica Divide y Vencerás

Características:

- Típicamente son usados cuando el tamaño de la entrada es significativamente grande y el problema puede ser dividido.
- Corresponde a un paradigma algorítmico basado en la recursividad o en resolver el mismo problema para entradas menores.
- Divide o particiona el problema en subproblemas más pequeños, que se parecen al original, los cuales resuelve independientemente para finalmente formar la solución.

Técnica Divide y Vencerás

Características:

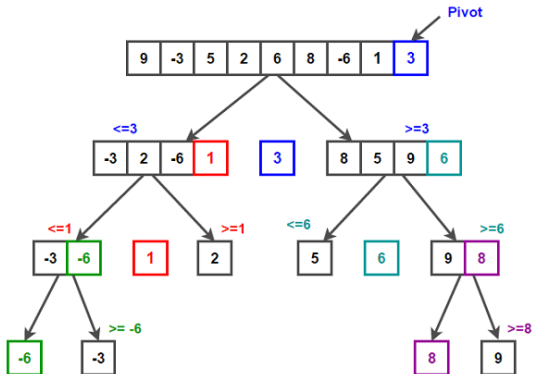
- Típicamente son usados cuando el tamaño de la entrada es significativamente grande y el problema puede ser dividido.
- Corresponde a un paradigma algorítmico basado en la recursividad o en resolver el mismo problema para entradas menores.
- Divide o particiona el problema en subproblemas más pequeños, que se parecen al original, los cuales resuelve independientemente para finalmente formar la solución.

Consta de tres etapas:

1. **Divide.** Se divide el problema en instancias más pequeñas del mismo problema.
2. **Vence.** Cada subproblema es resuelto de manera independiente y de forma local, donde es posible continuar aplicando subdivisiones a cada subproblema.
3. **Combina.** Las soluciones son combinadas para obtener la solución global al problema original.

QuickSort

Versión que elige como pivote al último elemento de la partición...



QuickSort

1. Selecciona un elemento como pivote (*piv*) —la versión básica toma siempre al primer o al último elemento como *piv*.
2. Partición, deja a la izquierda de *piv* los que son $\leq piv$ y a la derecha los mayores, retornando la posición final *p* de *piv*.
3. Recursivamente ordena cada partición.

Algoritmo quicksort(A, ℓ, r): ordena $A[\ell..r]$ ascendentemente.

Input: Arreglo $A[1..n]$, y sus límites $\ell = 1$ y $r = n$

Output: A ordenado

```
quicksort( $A, \ell, r$ ) {  
    if( $\ell < r$ ) then  
         $p = \text{partition}(A, \ell, r)$             $\rightarrow O(r - \ell) = \Theta(n)$   
        quicksort( $A, \ell, p - 1$ )              $\rightarrow T(p - \ell)$   
        quicksort( $A, p + 1, r$ )                $\rightarrow T(r - p)$   
}
```

QuickSort

1. Selecciona un elemento como pivote (*piv*) —la versión básica toma siempre al primer o al último elemento como *piv*.
2. Partición, deja a la izquierda de *piv* los que son $\leq piv$ y a la derecha los mayores, retornando la posición final *p* de *piv*.
3. Recursivamente ordena cada partición.

Algoritmo quicksort(A, ℓ, r): ordena $A[\ell..r]$ ascendentemente.

Input: Arreglo $A[1..n]$, y sus límites $\ell = 1$ y $r = n$

Output: A ordenado

```
quicksort( $A, \ell, r$ ) {  
  if( $\ell < r$ ) then  
     $p = \text{partition}(A, \ell, r)$        $\rightarrow O(r - \ell) = \Theta(n)$   
    quicksort( $A, \ell, p - 1$ )         $\rightarrow T(p - \ell)$   
    quicksort( $A, p + 1, r$ )          $\rightarrow T(r - p)$   
}
```

$\Rightarrow T(n) = \Theta(n) + T(m) + T(n - m)$, con $m \leq n$

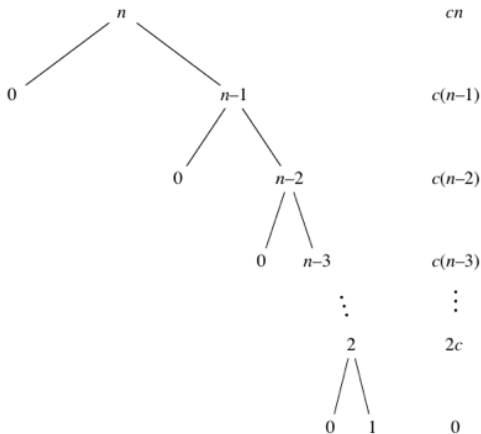
QuickSort

Versión que toma como pivote al primer elemento del subarreglo, $A[\ell]$.

```
partition( $A, \ell, r$ ) {  
     $piv = A[\ell]$   
     $p = \ell$   
    for( $i = \ell + 1$  to  $r$ ) do {  $\rightarrow O(r - \ell)$   
        if( $A[i] \leq piv$ ) then {  
             $p = p + 1$   
            swap( $A[i], A[p]$ )  
        }  
    }  
    swap( $A[\ell], A[p]$ )  
    return  $p$   
}  
 $\Rightarrow \text{partition}(A, \ell, r) \rightarrow \Theta(r - \ell)$ 
```

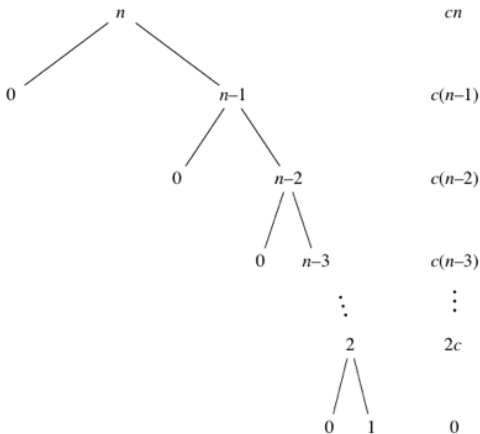
QuickSort - Worst Case

El peor caso ocurre cuando las particiones quedan totalmente desbalanceadas, es decir que en cada partición el pivote queda a un extremo del subarreglo particionado.



QuickSort - Worst Case

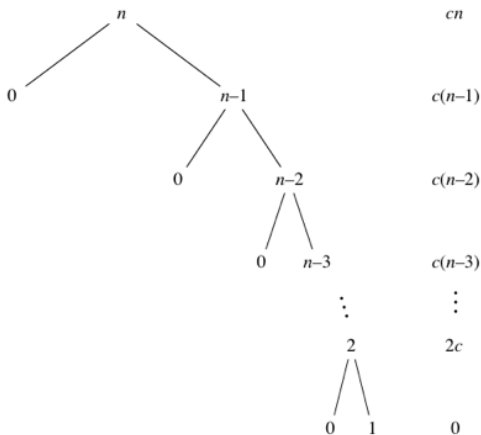
El **peor caso** ocurre cuando las particiones quedan totalmente desbalanceadas, es decir que en cada partición el pivote queda a un extremo del subarreglo particionado.



$$T(n) = \sum_{k=1}^n ck$$

QuickSort - Worst Case

El **peor caso** ocurre cuando las particiones quedan totalmente desbalanceadas, es decir que en cada partición el pivote queda a un extremo del subarreglo particionado.



$$T(n) = \sum_{k=1}^n ck = \Theta(n^2)$$

QuickSort - Worst Case

Otra forma es resolviendo directamente la recurrencia. Tenemos que:

$$\begin{aligned} T(n) &= \Theta(n) + T(m) + T(n - m), \text{ con } m = 1 \text{ y } T(1) = c \\ &= \Theta(n) + T(1) + T(n - 1) \end{aligned}$$

QuickSort - Worst Case

Otra forma es resolviendo directamente la recurrencia. Tenemos que:

$$T(n) = \Theta(n) + T(m) + T(n - m), \text{ con } m = 1 \text{ y } T(1) = c$$

$$= \Theta(n) + T(1) + T(n - 1)$$

$$= \Theta(n) + c + (\Theta(n - 1) + c + T(n - 2))$$

QuickSort - Worst Case

Otra forma es resolviendo directamente la recurrencia. Tenemos que:

$$T(n) = \Theta(n) + T(m) + T(n - m), \text{ con } m = 1 \text{ y } T(1) = c$$

$$= \Theta(n) + T(1) + T(n - 1)$$

$$= \Theta(n) + c + (\Theta(n - 1) + c + T(n - 2))$$

$$= \Theta(n) + \Theta(n - 1) + 2c + T(n - 2)$$

QuickSort - Worst Case

Otra forma es resolviendo directamente la recurrencia. Tenemos que:

$$T(n) = \Theta(n) + T(m) + T(n - m), \text{ con } m = 1 \text{ y } T(1) = c$$

$$= \Theta(n) + T(1) + T(n - 1)$$

$$= \Theta(n) + c + (\Theta(n - 1) + c + T(n - 2))$$

$$= \Theta(n) + \Theta(n - 1) + 2c + T(n - 2)$$

$$= \Theta(n) + \Theta(n - 1) + 2c + (\Theta(n - 2) + c + T(n - 3))$$

QuickSort - Worst Case

Otra forma es resolviendo directamente la recurrencia. Tenemos que:

$$T(n) = \Theta(n) + T(m) + T(n - m), \text{ con } m = 1 \text{ y } T(1) = c$$

$$= \Theta(n) + T(1) + T(n - 1)$$

$$= \Theta(n) + c + (\Theta(n - 1) + c + T(n - 2))$$

$$= \Theta(n) + \Theta(n - 1) + 2c + T(n - 2)$$

$$= \Theta(n) + \Theta(n - 1) + 2c + (\Theta(n - 2) + c + T(n - 3))$$

$$= \Theta(n) + \Theta(n - 1) + \Theta(n - 2) + 3c + T(n - 3)$$

QuickSort - Worst Case

Otra forma es resolviendo directamente la recurrencia. Tenemos que:

$$\begin{aligned}T(n) &= \Theta(n) + T(m) + T(n - m), \text{ con } m = 1 \text{ y } T(1) = c \\&= \Theta(n) + T(1) + T(n - 1) \\&= \Theta(n) + c + (\Theta(n - 1) + c + T(n - 2)) \\&= \Theta(n) + \Theta(n - 1) + 2c + T(n - 2) \\&= \Theta(n) + \Theta(n - 1) + 2c + (\Theta(n - 2) + c + T(n - 3)) \\&= \Theta(n) + \Theta(n - 1) + \Theta(n - 2) + 3c + T(n - 3) \\&\dots \\&= \Theta(n) + \Theta(n - 1) + \dots + \Theta(n - (k - 1)) + kc + T(n - k)\end{aligned}$$

QuickSort - Worst Case

Otra forma es resolviendo directamente la recurrencia. Tenemos que:

$$T(n) = \Theta(n) + T(m) + T(n - m), \text{ con } m = 1 \text{ y } T(1) = c$$

$$= \Theta(n) + T(1) + T(n - 1)$$

$$= \Theta(n) + c + (\Theta(n - 1) + c + T(n - 2))$$

$$= \Theta(n) + \Theta(n - 1) + 2c + T(n - 2)$$

$$= \Theta(n) + \Theta(n - 1) + 2c + (\Theta(n - 2) + c + T(n - 3))$$

$$= \Theta(n) + \Theta(n - 1) + \Theta(n - 2) + 3c + T(n - 3)$$

...

$$= \Theta(n) + \Theta(n - 1) + \dots + \Theta(n - (k - 1)) + kc + T(n - k)$$

$$\text{Si } k = n - 1 \Rightarrow \Theta(n - (k - 1)) + kc + T(n - k) = \Theta(2) + (n - 1)c + T(1)$$

QuickSort - Worst Case

Otra forma es resolviendo directamente la recurrencia. Tenemos que:

$$T(n) = \Theta(n) + T(m) + T(n - m), \text{ con } m = 1 \text{ y } T(1) = c$$

$$= \Theta(n) + T(1) + T(n - 1)$$

$$= \Theta(n) + c + (\Theta(n - 1) + c + T(n - 2))$$

$$= \Theta(n) + \Theta(n - 1) + 2c + T(n - 2)$$

$$= \Theta(n) + \Theta(n - 1) + 2c + (\Theta(n - 2) + c + T(n - 3))$$

$$= \Theta(n) + \Theta(n - 1) + \Theta(n - 2) + 3c + T(n - 3)$$

...

$$= \Theta(n) + \Theta(n - 1) + \dots + \Theta(n - (k - 1)) + kc + T(n - k)$$

$$\text{Si } k = n - 1 \Rightarrow \Theta(n - (k - 1)) + kc + T(n - k) = \Theta(2) + (n - 1)c + T(1)$$

$$\Rightarrow T(n) = \sum_{k=2}^n \Theta(k) + (n - 1)c + c, \text{ con } T(1) = c$$

QuickSort - Worst Case

Otra forma es resolviendo directamente la recurrencia. Tenemos que:

$$T(n) = \Theta(n) + T(m) + T(n - m), \text{ con } m = 1 \text{ y } T(1) = c$$

$$= \Theta(n) + T(1) + T(n - 1)$$

$$= \Theta(n) + c + (\Theta(n - 1) + c + T(n - 2))$$

$$= \Theta(n) + \Theta(n - 1) + 2c + T(n - 2)$$

$$= \Theta(n) + \Theta(n - 1) + 2c + (\Theta(n - 2) + c + T(n - 3))$$

$$= \Theta(n) + \Theta(n - 1) + \Theta(n - 2) + 3c + T(n - 3)$$

...

$$= \Theta(n) + \Theta(n - 1) + \dots + \Theta(n - (k - 1)) + kc + T(n - k)$$

$$\text{Si } k = n - 1 \Rightarrow \Theta(n - (k - 1)) + kc + T(n - k) = \Theta(2) + (n - 1)c + T(1)$$

$$\Rightarrow T(n) = \sum_{k=2}^n \Theta(k) + (n - 1)c + c, \text{ con } T(1) = c$$

$$\leq c' \frac{n(n+1)}{2} + nc$$

QuickSort - Worst Case

Otra forma es resolviendo directamente la recurrencia. Tenemos que:

$$T(n) = \Theta(n) + T(m) + T(n - m), \text{ con } m = 1 \text{ y } T(1) = c$$

$$= \Theta(n) + T(1) + T(n - 1)$$

$$= \Theta(n) + c + (\Theta(n - 1) + c + T(n - 2))$$

$$= \Theta(n) + \Theta(n - 1) + 2c + T(n - 2)$$

$$= \Theta(n) + \Theta(n - 1) + 2c + (\Theta(n - 2) + c + T(n - 3))$$

$$= \Theta(n) + \Theta(n - 1) + \Theta(n - 2) + 3c + T(n - 3)$$

...

$$= \Theta(n) + \Theta(n - 1) + \dots + \Theta(n - (k - 1)) + kc + T(n - k)$$

$$\text{Si } k = n - 1 \Rightarrow \Theta(n - (k - 1)) + kc + T(n - k) = \Theta(2) + (n - 1)c + T(1)$$

$$\Rightarrow T(n) = \sum_{k=2}^n \Theta(k) + (n - 1)c + c, \text{ con } T(1) = c$$

$$\leq c' \frac{n(n+1)}{2} + nc = \Theta(n^2), \text{ con } c', c \text{ constantes.}$$

QuickSort - Best Case

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo a particionar; es decir, cuando el árbol de recurrencia queda perfectamente balanceado.

QuickSort - Best Case

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo a particionar; es decir, cuando el árbol de recurrencia queda perfectamente balanceado.

$$\Rightarrow T(n) = \Theta(n) + 2T(n/2)$$

QuickSort - Best Case

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo a particionar; es decir, cuando el árbol de recurrencia queda perfectamente balanceado.

$$\Rightarrow T(n) = \Theta(n) + 2T(n/2)$$

- Usando el teorema maestro, tenemos que obedece al caso:

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n), \text{ con } a = b = 2$$

QuickSort - Best Case

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo a particionar; es decir, cuando el árbol de recurrencia queda perfectamente balanceado.

$$\Rightarrow T(n) = \Theta(n) + 2T(n/2)$$

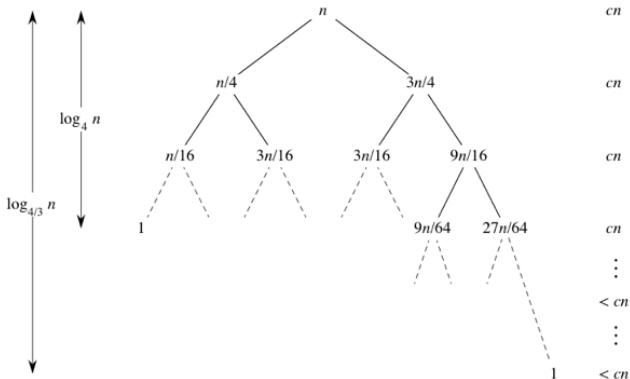
- Usando el teorema maestro, tenemos que obedece al caso:

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n), \text{ con } a = b = 2$$

$$\therefore T(n) = \Theta(n \log n)$$

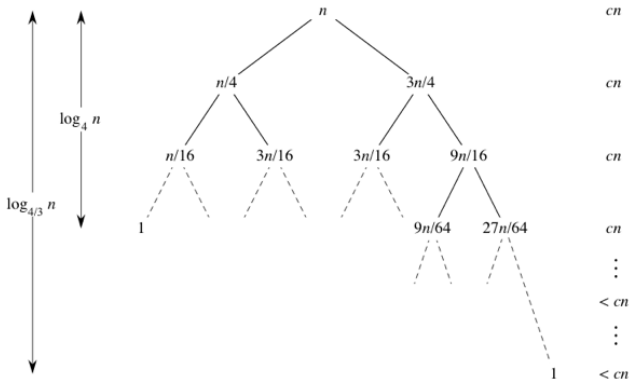
QuickSort - Average Case

El caso promedio ocurre cuando las particiones alternadamente quedan entre perfectamente balanceadas y totalmente desbalanceadas. Es decir oscilan entre $(\frac{n}{2}, \frac{n}{2})$ y $(1, n-1)$ con igual probabilidad, siendo el promedio $(\frac{n}{4}, \frac{3n}{4})$



QuickSort - Average Case

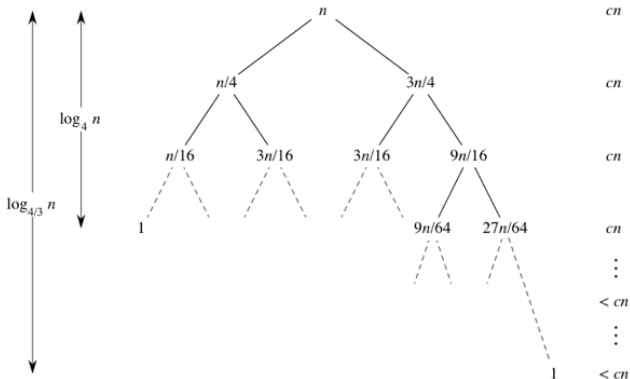
El caso promedio ocurre cuando las particiones alternadamente quedan entre perfectamente balanceadas y totalmente desbalanceadas. Es decir oscilan entre $(\frac{n}{2}, \frac{n}{2})$ y $(1, n-1)$ con igual probabilidad, siendo el promedio $(\frac{n}{4}, \frac{3n}{4})$



$$\Rightarrow \log_4 n(c \cdot n) \leq T(n) \leq \log_{4/3} n(c \cdot n)$$

QuickSort - Average Case

El caso promedio ocurre cuando las particiones alternadamente quedan entre perfectamente balanceadas y totalmente desbalanceadas. Es decir oscilan entre $(\frac{n}{2}, \frac{n}{2})$ y $(1, n-1)$ con igual probabilidad, siendo el promedio $(\frac{n}{4}, \frac{3n}{4})$

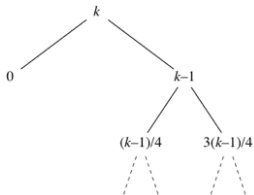


$$\Rightarrow \log_4 n(c \cdot n) \leq T(n) \leq \log_{4/3} n(c \cdot n)$$

$$\therefore T(n) = \Theta(n \log n)$$

QuickSort - Average Case

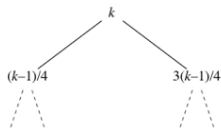
Otra forma de argumentar que el caso promedio es $\Theta(n \log n)$ es pensando en el peor caso promedio posible. Este ocurre cuando de forma alternada hacemos una peor partición $(1, n - 1)$ seguida de una peor partición dentro del promedio $(\frac{n}{4}, \frac{3n}{4})$. entonces:



ck

$c(k-1)$

$c(k-1)$

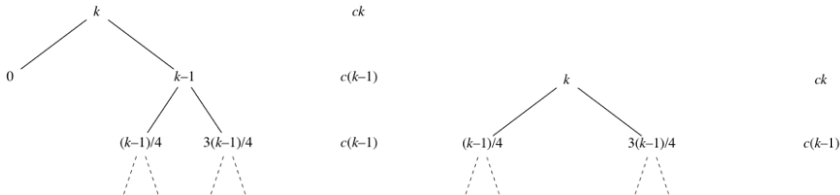


ck

$c(k-1)$

QuickSort - Average Case

Otra forma de argumentar que el caso promedio es $\Theta(n \log n)$ es pensando en el peor caso promedio posible. Este ocurre cuando de forma alternada hacemos una peor partición $(1, n - 1)$ seguida de una peor partición dentro del promedio $(\frac{n}{4}, \frac{3n}{4})$. entonces:



Entonces tenemos que en este peor de los casos promedio, la altura del árbol no puede ser mayor al doble que la altura del árbol anterior en su peor caso; es decir $h' \leq 2h = 2 \log_{4/3} n$; amplificando el costo por un valor constante que no interesea en el análisis asintótico de nuestra notación, afirmando que $T(n) = \Theta(n \log n)$

Algoritmo QuickSelect

- Es un algoritmo de Selección, que encuentra el k -ésimo menor elemento de una lista desordenada.

Algoritmo QuickSelect

- Es un algoritmo de Selección, que encuentra el k -ésimo menor elemento de una lista desordenada.
- Se basa en el principio de ordenamiento de QuickSort.

Algoritmo QuickSelect

- Es un algoritmo de Selección, que encuentra el k -ésimo menor elemento de una lista desordenada.
- Se basa en el principio de ordenamiento de QuickSort. $\Rightarrow$ Es bueno en la práctica (en promedio) pero con un mal peor caso.

Algoritmo QuickSelect

- Es un algoritmo de Selección, que encuentra el k -ésimo menor elemento de una lista desordenada.
- Se basa en el principio de ordenamiento de QuickSort. $\Rightarrow$ Es bueno en la práctica (en promedio) pero con un mal peor caso.

Algoritmo:

1. Selecciona un elemento como pivote (piv)
2. Partición. Deja a la izquierda de piv los que son $\leq piv$ y a la derecha los mayores, retornando la posición p de piv .
3. Si el elemento buscado es piv termina ($p = k$); sino selecciona la partición en donde debe estar el k -ésimo menor y aplica recursividad solo en esta partición actualizando k si es necesario.

Algoritmo QuickSelect

Algoritmo quickSelect(A, ℓ, r, k): encuentra el k -ésimo menor en $A[\ell..r]$

Input: Arreglo $A[1..n]$, k y sus límites $\ell = 1$ y $r = n$

Output: k -ésimo menor

```
quickSelect( $A, \ell, r, k$ ){  
    if ( $\ell = r$ ) then  
        return  $A[\ell]$   
     $p = \text{partition}(A, \ell, r)$   
     $u = p - \ell$  // # celdas a la izquierda de  $p$   
    if ( $k = u + 1$ ) then  
        return  $A[p]$  // el pivote es el buscado  
    if ( $k \leq u$ ) then  
        return quickSelect( $A, \ell, p - 1, k$ ) // el  $k$ -ésimo menor está a la izquierda  
    return quickSelect( $A, p + 1, r, k - u - 1$ ) // el  $k$ -ésimo menor está a la derecha  
}
```

Algoritmo QuickSelect

Algoritmo quickSelect(A, ℓ, r, k): encuentra el k -ésimo menor en $A[\ell..r]$

Input: Arreglo $A[1..n]$, k y sus límites $\ell = 1$ y $r = n$

Output: k -ésimo menor

```
quickSelect( $A, \ell, r, k$ ){  
    if ( $\ell = r$ ) then  
        return  $A[\ell]$   
     $p = \text{partition}(A, \ell, r)$   
     $u = p - \ell$  // # celdas a la izquierda de  $p$   
    if ( $k = u + 1$ ) then  
        return  $A[p]$  // el pivote es el buscado  
    if ( $k \leq u$ ) then  
        return quickSelect( $A, \ell, p - 1, k$ ) // el  $k$ -ésimo menor está a la izquierda  
    return quickSelect( $A, p + 1, r, k - u - 1$ ) // el  $k$ -ésimo menor está a la derecha  
}
```

El peor caso, al igual que QuickSort, es cuando el pivote queda siempre en $A[\ell]$ o $A[r]$

Algoritmo QuickSelect

Algoritmo quickSelect(A, ℓ, r, k): encuentra el k -ésimo menor en $A[\ell..r]$

Input: Arreglo $A[1..n]$, k y sus límites $\ell = 1$ y $r = n$

Output: k -ésimo menor

```
quickSelect( $A, \ell, r, k$ ) {  
    if ( $\ell = r$ ) then  
        return  $A[\ell]$   
     $p = \text{partition}(A, \ell, r)$   
     $u = p - \ell$  // # celdas a la izquierda de  $p$   
    if ( $k = u + 1$ ) then  
        return  $A[p]$  // el pivote es el buscado  
    if ( $k \leq u$ ) then  
        return quickSelect( $A, \ell, p - 1, k$ ) // el  $k$ -ésimo menor está a la izquierda  
    return quickSelect( $A, p + 1, r, k - u - 1$ ) // el  $k$ -ésimo menor está a la derecha  
}
```

El peor caso, al igual que QuickSort, es cuando el pivote queda siempre en $A[\ell]$ o $A[r]$

$$\Rightarrow T(n) = \Theta(n) + T(n - 1)$$

Algoritmo QuickSelect

Algoritmo quickSelect(A, ℓ, r, k): encuentra el k -ésimo menor en $A[\ell..r]$

Input: Arreglo $A[1..n]$, k y sus límites $\ell = 1$ y $r = n$

Output: k -ésimo menor

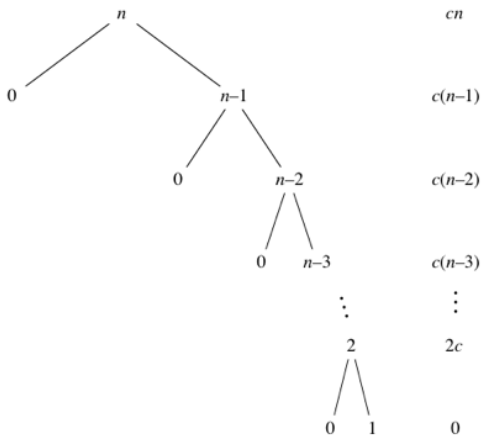
```
quickSelect( $A, \ell, r, k$ ) {  
    if ( $\ell = r$ ) then  
        return  $A[\ell]$   
     $p = \text{partition}(A, \ell, r)$  // # celdas a la izquierda de  $p$   
     $u = p - \ell$   
    if ( $k = u + 1$ ) then  
        return  $A[p]$  // el pivote es el buscado  
    if ( $k \leq u$ ) then  
        return quickSelect( $A, \ell, p - 1, k$ ) // el  $k$ -ésimo menor está a la izquierda  
    return quickSelect( $A, p + 1, r, k - u - 1$ ) // el  $k$ -ésimo menor está a la derecha  
}
```

El peor caso, al igual que QuickSort, es cuando el pivote queda siempre en $A[\ell]$ o $A[r]$

$\Rightarrow T(n) = \Theta(n) + T(n - 1) = \Theta(n^2)$, al igual que quicksort.

Recuerde: QuickSort - Worst Case

El **peor caso** ocurre cuando las particiones quedan totalmente desbalanceadas, es decir que en cada partición el pivote queda a un extremo del subarreglo particionado.



$$T(n) = \sum_{k=1}^n ck = \Theta(n^2)$$

Algoritmo QuickSelect

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo particionado.

Algoritmo QuickSelect

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo particionado.

$$\Rightarrow T(n) = \Theta(n) + T(n/2)$$

Algoritmo QuickSelect

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo particionado.

$$\Rightarrow T(n) = \Theta(n) + T(n/2)$$

Usando el teorema maestro, tenemos que obedece al caso:

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n), \text{ con } a = 1, b = 2 \text{ y } f(n) = \Theta(n)$$

Algoritmo QuickSelect

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo particionado.

$$\Rightarrow T(n) = \Theta(n) + T(n/2)$$

Usando el teorema maestro, tenemos que obedece al caso:

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n), \text{ con } a = 1, b = 2 \text{ y } f(n) = \Theta(n)$$

$$\Rightarrow T(n) = \Theta(n)$$

- **El caso promedio** ocurre cuando las particiones alternadamente quedan entre perfectamente balanceadas y totalmente desbalanceadas y el k -ésimo menor está en la partición mayor. Es decir oscilan entre $(\frac{n}{2}, \frac{n}{2})$ y $(1, n-1)$ con igual probabilidad, siendo en promedio $(\frac{n}{4}, \frac{3n}{4})$.

Algoritmo QuickSelect

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo particionado.

$$\Rightarrow T(n) = \Theta(n) + T(n/2)$$

Usando el teorema maestro, tenemos que obedece al caso:

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n), \text{ con } a = 1, b = 2 \text{ y } f(n) = \Theta(n)$$

$$\Rightarrow T(n) = \Theta(n)$$

- **El caso promedio** ocurre cuando las particiones alternadamente quedan entre perfectamente balanceadas y totalmente desbalanceadas y el k -ésimo menor está en la partición mayor. Es decir oscilan entre $(\frac{n}{2}, \frac{n}{2})$ y $(1, n-1)$ con igual probabilidad, siendo en promedio $(\frac{n}{4}, \frac{3n}{4})$. Suponga que siempre el buscado está en la partición de tamaño $\frac{3n}{4}$

Algoritmo QuickSelect

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo particionado.

$$\Rightarrow T(n) = \Theta(n) + T(n/2)$$

Usando el teorema maestro, tenemos que obedece al caso:

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n), \text{ con } a = 1, b = 2 \text{ y } f(n) = \Theta(n)$$

$$\Rightarrow T(n) = \Theta(n)$$

- **El caso promedio** ocurre cuando las particiones alternadamente quedan entre perfectamente balanceadas y totalmente desbalanceadas y el k -ésimo menor está en la partición mayor. Es decir oscilan entre $(\frac{n}{2}, \frac{n}{2})$ y $(1, n-1)$ con igual probabilidad, siendo en promedio $(\frac{n}{4}, \frac{3n}{4})$.

Suponga que siempre el buscado está en la partición de tamaño $\frac{3n}{4}$

$$\Rightarrow T(n) = \Theta(n) + T\left(\frac{3n}{4}\right)$$

Algoritmo QuickSelect

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo particionado.

$$\Rightarrow T(n) = \Theta(n) + T(n/2)$$

Usando el teorema maestro, tenemos que obedece al caso:

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n), \text{ con } a = 1, b = 2 \text{ y } f(n) = \Theta(n)$$

$$\Rightarrow T(n) = \Theta(n)$$

- **El caso promedio** ocurre cuando las particiones alternadamente quedan entre perfectamente balanceadas y totalmente desbalanceadas y el k -ésimo menor está en la partición mayor. Es decir oscilan entre $(\frac{n}{2}, \frac{n}{2})$ y $(1, n-1)$ con igual probabilidad, siendo en promedio $(\frac{n}{4}, \frac{3n}{4})$.

Suponga que siempre el buscado está en la partición de tamaño $\frac{3n}{4}$

$$\Rightarrow T(n) = \Theta(n) + T\left(\frac{3n}{4}\right) \Rightarrow \text{aplicando TM con } a = 1 < b = 4/3$$

Algoritmo QuickSelect

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo particionado.

$$\Rightarrow T(n) = \Theta(n) + T(n/2)$$

Usando el teorema maestro, tenemos que obedece al caso:

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n), \text{ con } a = 1, b = 2 \text{ y } f(n) = \Theta(n)$$

$$\Rightarrow T(n) = \Theta(n)$$

- **El caso promedio** ocurre cuando las particiones alternadamente quedan entre perfectamente balanceadas y totalmente desbalanceadas y el k -ésimo menor está en la partición mayor. Es decir oscilan entre $(\frac{n}{2}, \frac{n}{2})$ y $(1, n-1)$ con igual probabilidad, siendo en promedio $(\frac{n}{4}, \frac{3n}{4})$.

Suponga que siempre el buscado está en la partición de tamaño $\frac{3n}{4}$

$$\Rightarrow T(n) = \Theta(n) + T\left(\frac{3n}{4}\right) \Rightarrow \text{aplicando TM con } a = 1 < b = 4/3$$

$$\Rightarrow T(n) = \Theta(n)$$

Algoritmo QuickSelect

- **El mejor caso** ocurre cuando el pivote queda siempre en el centro del subarreglo particionado.

$$\Rightarrow T(n) = \Theta(n) + T(n/2)$$

Usando el teorema maestro, tenemos que obedece al caso:

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n), \text{ con } a = 1, b = 2 \text{ y } f(n) = \Theta(n)$$

$$\Rightarrow T(n) = \Theta(n)$$

- **El caso promedio** ocurre cuando las particiones alternadamente quedan entre perfectamente balanceadas y totalmente desbalanceadas y el k -ésimo menor está en la partición mayor. Es decir oscilan entre $(\frac{n}{2}, \frac{n}{2})$ y $(1, n-1)$ con igual probabilidad, siendo en promedio $(\frac{n}{4}, \frac{3n}{4})$.

Suponga que siempre el buscado está en la partición de tamaño $\frac{3n}{4}$

$$\Rightarrow T(n) = \Theta(n) + T\left(\frac{3n}{4}\right) \Rightarrow \text{aplicando TM con } a = 1 < b = 4/3$$

$$\Rightarrow T(n) = \Theta(n \log_{4/3} n)$$

¿Porqué el costo de QuickSelect, en el caso promedio, no es $\Theta(n \log n)$ si la altura es $\log_{4/3} n$ al igual que QuickSort?

